A Schmitt-Trigger-Assisted 4×4 SRAM PUF Array for IoT Hardware Security

Karim Taha, Osama Zidan
Department of Electrical and Computer Engineering
Birzeit University, Birzeit, Palestine
Digital Integrated Circuits Course Project

***Abstract*—A small hardware "fingerprint" circuit, called a Physically Unclonable Function (PUF), was designed. A unique secret key is generated for each chip based on random manufacturing differences, without requiring the key to be stored in memory. A standard 6T SRAM cell was used as the PUF bit, and a Schmitt trigger was added to enable cleaner readout. The design was implemented in Electric VLSI and verified using SPICE simulation. The obtained results were: $V_{DD} = 1.8$ V, average power ≈0.58 μW, peak power ≈280 μW, total layout area ≈0.154 mm^2, and DRC-clean layout. The results were compared with the ASCH-PUF paper [1].**

***Index Terms*—PUF, SRAM, Schmitt trigger, IoT security, Electric VLSI**

I. Introduction

IoT chips need a secret key for security, but normal memory (NVM) can be read or probed by attackers. A PUF solves this differently: it uses random transistor differences from manufacturing to create a key that only exists while the chip is powered on. No key is ever stored.

This project is based on the ASCH-PUF paper [1], which builds a large, fabricated PUF chip with self-checking hardware. We do not copy that design. Instead, we build a much smaller version for a class project: a 6T SRAM PUF cell with a Schmitt trigger for cleaner readout, plus a simple way to detect unstable bits.

Question we are answering: Can a small SRAM PUF be more reliable using a Schmitt trigger and simple repeated-read testing, without needing big error-correction hardware?

II. Background

A PUF is tested once to record a "golden" answer, then tested again later to see if it repeats. If a bit changes, that is an error (BER = bit error rate). Designers either fix errors with error-correction code (costly) or find and remove unstable bits ("masking").

The ASCH-PUF paper [1] finds and fixes unstable bits using a voltage-skew trick and on-chip healing circuits. It reaches almost 0 BER with only 31–35% of bits masked, in a real 65 nm chip.

Their skewing and healing circuits were not implemented. Instead, two main ideas were retained: (1) a simple PUF cell was built from basic digital gates, and (2) each bit was checked more than once to identify unstable bits, rather than using full temperature tests.

III. Circuit Design and Layout

This section covers each building block once: what it does, its schematic, and its physical layout, together.

A. CMOS Inverter

The basic building block: NMOS width $W = 5$, PMOS width $W = 10$, sized roughly 2:1 to balance electron/hole mobility so it switches near half of V_{DD}. It is used inside the SRAM cell and the Schmitt trigger, so it has no separate schematic or layout of its own; it appears inside the figures below.

B. 6T SRAM PUF Cell

Two cross-coupled inverters plus two access transistors. At power-up, tiny manufacturing differences push the cell to a random "0" or "1". The same cell always gives the same answer on the same chip, but different chips give different answers. This randomness is the PUF.

Table I shows how the cell behaves. Hold, Read, and both Write cases were tested in SPICE. The last row (BL = BLB) is an illegal state, avoided by design.

Fig. 1 shows the schematic of one cell, using the corrected $W = 10$ pull-down transistors (Section III-C). WL gates both access transistors together. When WL is high, BL and _BL connect to the two cross-coupled inverters that store the bit.

TABLE I: 6T SRAM Cell Truth Table

WL	BL	BLB	Op.	Result
0	X	X	Hold	Q unchanged
1	pre. V_{DD}	pre. V_{DD}	Read	Q unchanged
1	strong 1	strong 0	Write 1	$Q \rightarrow 1$
1	strong 0	strong 1	Write 0	$Q \rightarrow 0$
1	BL = BLB		Illegal	avoid

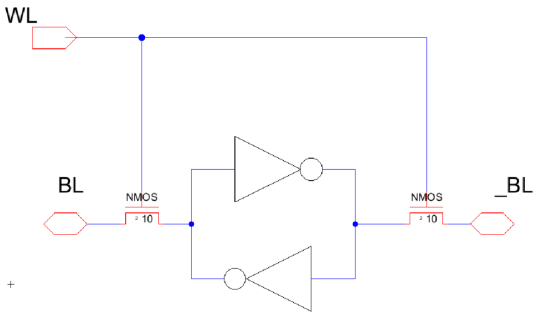


Fig. 1: 1-bit PUF cell schematic: access transistors (WL, BL, _BL) plus the cross-coupled inverter pair.

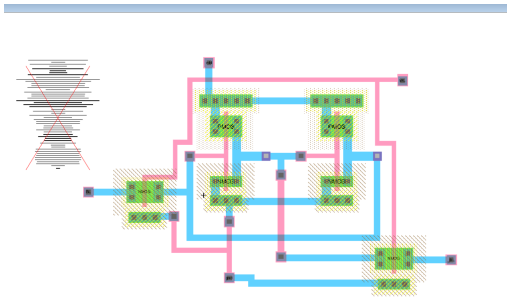


Fig. 2: 1-bit PUF cell layout: PMOS pull-up pair on top, cross-coupled NMOS pull-down pair in the middle, WL-gated access transistors (BL, _BL) on the sides.

When WL is low, the access transistors turn off and the cell simply holds its value.

Fig. 2 shows the matching physical layout. The two PMOS pull-up transistors sit on top, the two cross-coupled NMOS pull-down transistors sit in the middle, and the two WL-gated access transistors sit on the sides. The cross-coupled pair is mirrored around a shared axis to keep the two halves physically matched, so randomness comes from real manufacturing variation and not from a lopsided layout.

C. Sizing Fix

At first, the pull-down NMOS in the cell used $W = 5$, the same as our normal inverter. This was too weak: the cell could not pull its storage node fully low when read through the Schmitt trigger, so the output got stuck near the middle instead of a clean logic level.

Fix: the pull-down width was doubled to $W = 10$, everywhere in the array. After the fix, the node discharges cleanly and the output no longer gets stuck (see Fig. 12).

Fig. 3 shows this discharge directly on the cell's own internal nodes. `net@85` and `net@89` are the two storage

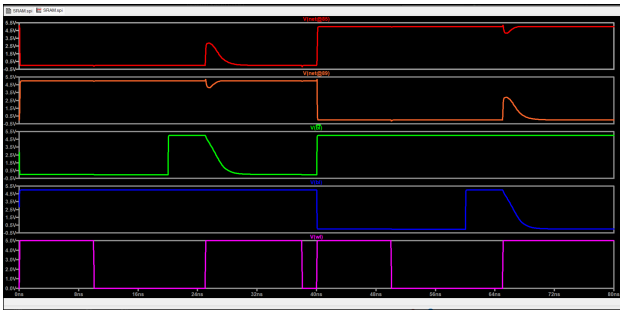


Fig. 3: SRAM cell internal nodes (`net@85`, `net@89`), BL, _BL, and WL over several access cycles, with $W = 10$ pull-down sizing. Both nodes discharge fully and repeatably.

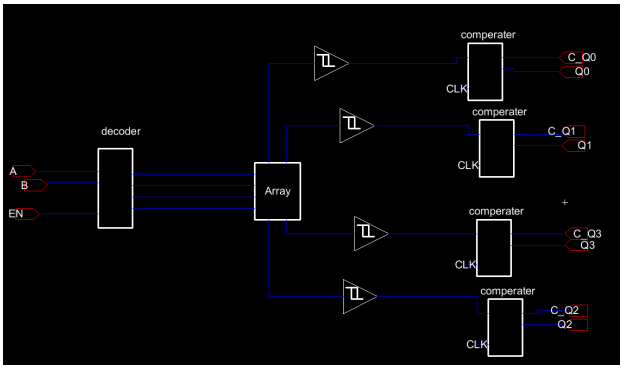


Fig. 4: PUF cell with Schmitt-trigger readout.

nodes of the cross-coupled pair. Each time WL pulses and BL or _BL is driven, one node pulls all the way down to 0 V and the other stays high near V_{DD}, with no stall in the middle. This repeats cleanly across several write/read cycles in the same run, showing the fix holds every time the cell is accessed, not just once.

D. Schmitt-Trigger Readout

Fig. 4 shows the PUF cell with its Schmitt-trigger readout at the system level. The Schmitt trigger adds hysteresis, which gives a wider, safer switching zone. Cells with very small mismatch, which would otherwise give a shaky answer, get pushed more firmly to a clean 0 or 1.

Fig. 5 shows the physical layout of the Schmitt-trigger stage, built from the same corrected ($W = 10$) transistor sizing as the SRAM cell.

Fig. 6 shows the actual transistor-level Schmitt trigger, with its own testbench. It is built from two stacked inverter paths: a main PMOS/NMOS pair sized W = 20/10, plus a smaller feedback PMOS/NMOS pair sized W = 4/4. The feedback pair only turns on after the output has already switched, creating hysteresis: it makes the trip point different depending on whether the input is rising or falling, so small, noisy inputs near the middle do not cause false switching.

E. Reading the Array

One row is selected at a time using a 2×4 decoder. All 4 columns of that row are read at the same time, each through its

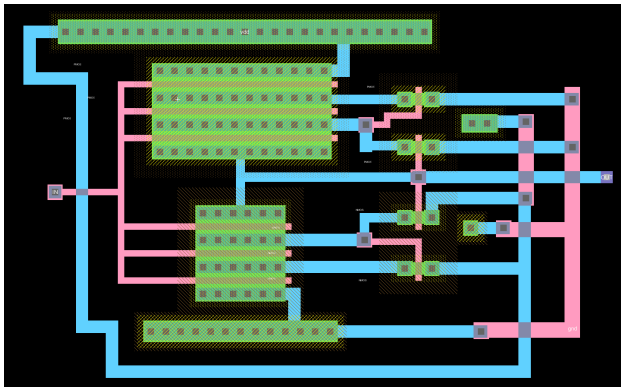


Fig. 5: Schmitt-trigger layout

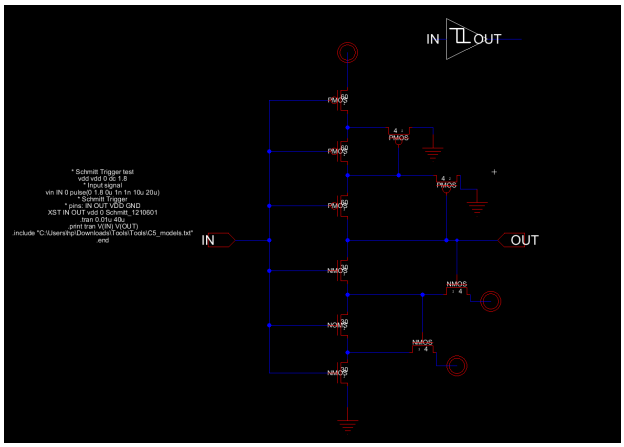


Fig. 6: Schmitt-trigger transistor-level schematic and testbench. Main pair: $W = 20$ (PMOS) / $W = 10$ (NMOS). Feedback pair: $W = 4$ / $W = 4$.

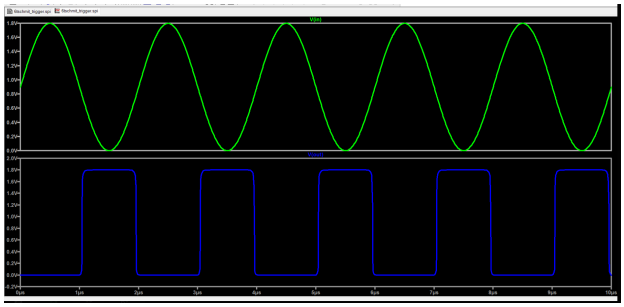


Fig. 7: Schmitt-trigger transistor-level layout (IN/OUT pins, V_{DD}/GND rails).

own Schmitt trigger and comparator/latch. Reading all 4 rows in sequence covers the full 16-bit array. No column multiplexer is needed at this size.

Fig. 8 shows the physical layout of the full array, built from 16 copies of the same corrected cell. **DRC: clean, no violations.**

F. Unstable-Bit Detection

Instead of full temperature/voltage sweeps like [1], each cell is simply read more than once. If a bit ever flips between reads, it is flagged as unstable and dropped from the final key.

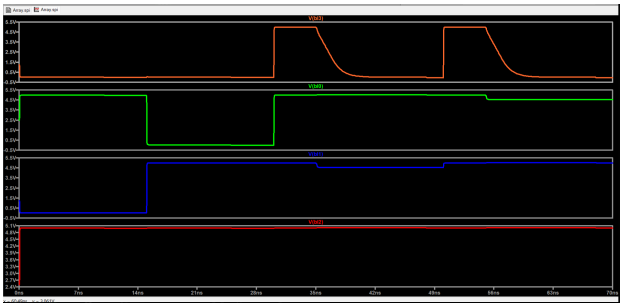


Fig. 8: 4×4 PUF array layout.

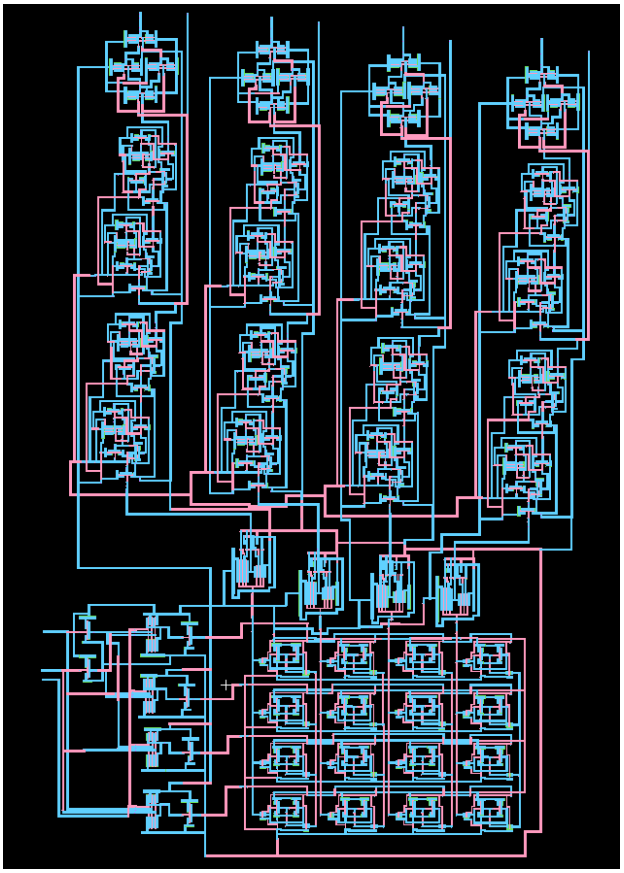


Fig. 9: Full macro layout (array + decoder + Schmitt triggers + comparators), used for the area measurement.

G. Layout Area

Fig. 9 shows the full macro layout: array, decoder, Schmitt-trigger stages, and comparators together. This is what was measured for the area calculation below.

Measured bounding box: 1672×2303.5 grid units, at $0.2\,\mu$m/unit:

$$334.4\,\mu\text{m} \times 460.7\,\mu\text{m} \approx 0.154\,\text{mm}^2$$

This covers the array, decoder, Schmitt triggers, and comparators together (16 bits $\Rightarrow\approx$9,629 μm^2/bit).

Vs. ASCH-PUF [1]: their bit-cell is only 2.51 μm^2 (594 F^2) in a real 65 nm process, with a full 4096-bit array at just 0.018 mm^2. Our per-bit area is about 2,190× larger.

TABLE II: Area: This Work vs. ASCH-PUF

Metric	This Work	ASCH-PUF
Process	Generic 0.2 μm	65 nm CMOS
Bits	16	4096
Total area	0.154 mm^2	0.018 mm^2
Area/bit	9,629 μm^2	4.39 μm^2

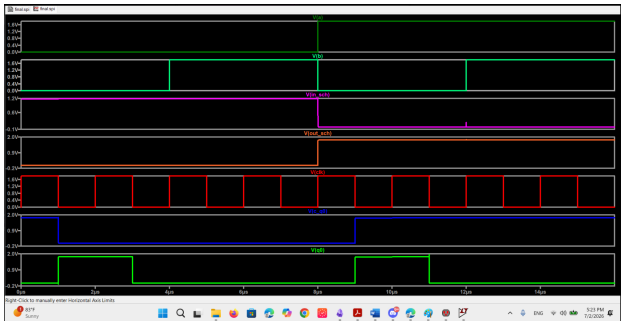
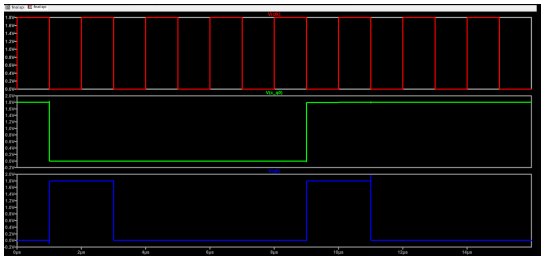
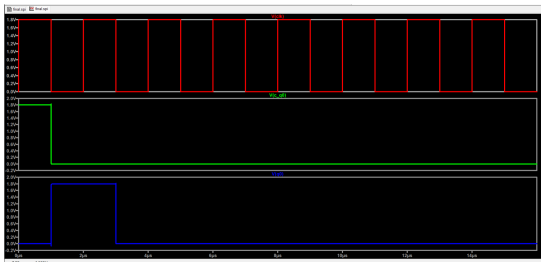


Fig. 10: Power-up waveforms.



(a) Test 1: same cell, repeated power-up. Stable output.



(b) Test 2: independent re-run, same file. Different output.

Fig. 11: Test 1 (top) vs. Test 2 (bottom): stable on repeat, different on an independent re-run.

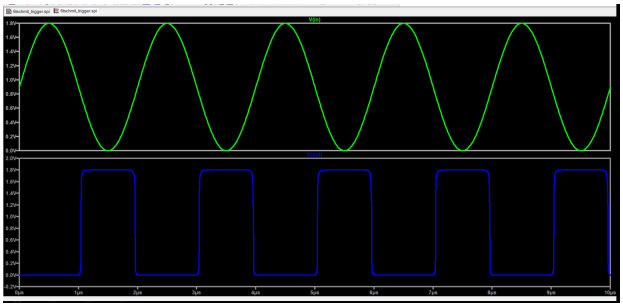


Fig. 12: Schmitt-trigger output after the sizing fix.

Three reasons: (1) our process grid is much coarser (0.4 μm vs. 65 nm, roughly a 38$\times$ area penalty from lithography alone), (2) our layout is not compacted, and (3) our array is tiny, so support circuits are not spread across many bits like theirs are.

IV. Simulation Results

$V_{DD} = 1.8$ V for all tests.

A. Power-Up

Fig. 10 shows the cell resolving from an unclear middle voltage to a stable 0 or 1 as power turns on, driven by transistor mismatch.

B. Speed

The PUF resolves each row in well under 4 μs. `out_sch` settles almost immediately after the row address changes, and Q_0/C_{Q0} update right at the next CLK edge. Read access and clock-to-Q delay are both fast relative to the 4 μs row window.

C. Test 1 and Test 2: Stable vs. Random

A good PUF must be both **stable** (same cell, same answer every time) and **random** (different instances give different answers). Fig. 11 stacks both tests together for direct comparison. The same simulated cell, powered up and read repeatedly, always settles to the same value (top). The identical testbench file, run again as a fresh, independent simulation with no design change, settles to a different value (bottom).

Why does Test 2 give a different answer? The cross-coupled pair is symmetric, so at power-up it sits on a perfect balance point with no natural push either way. The simulator log shows this directly: normal solving fails ("Direct Newton iteration failed"), and the solver falls back to a step-by-step method (Gmin-stepping) to find an answer. That method is sensitive to tiny rounding differences between runs. No manual changes were made between runs; the different results came from this solver behavior. It acts like real transistor mismatch, but it is not the same thing as an explicit mismatch model. See Section VI for next steps.

D. Schmitt-Trigger Behavior

Fig. 12 shows clean switching with hysteresis after the $W = 10$ fix. Noise on the signal is clearly smaller after the Schmitt trigger than before it, which is exactly what it is meant to do. The small ripple still visible on comparator outputs (for example, C_{Q0}) reflects this attenuated noise floor, not a wiring fault.

E. Array-Level Power-Up

Fig. 13 shows the whole array powering up correctly with the fixed sizing. The fix works array-wide, not just for one cell. Each bitline is precharged high, then pulled down only when its column is written or read, and returns cleanly to the precharge level afterward: the same clean discharge behavior seen in Fig. 3, now confirmed across the whole array.

F. Readout: Q_0–Q_3

The decoder selects one row at a time; all 4 columns of that row are read together. Four row-reads cover all 16 bits.

Fig. 13: Array-level power-up.

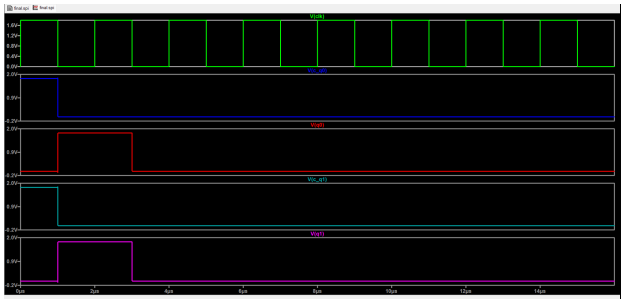


Fig. 14: Q_0/Q_1 readout.

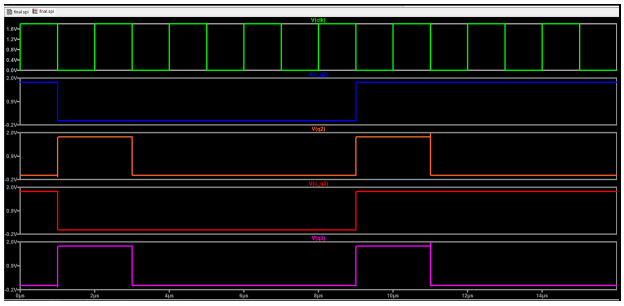


Fig. 15: Q_2/Q_3 readout.

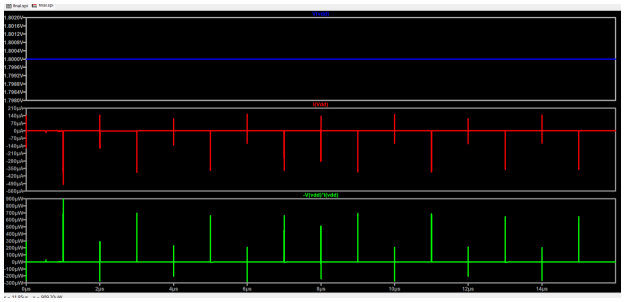


Fig. 16: Supply current/power waveform.

Figs. 14–15 show clean, distinct, repeatable outputs for each bit.

G. Power and Energy

Fig. 16 and Fig. 17 show current draw and the SPICE power summary. The circuit idles under 1 μW and spikes to a few hundred μW at each clock edge. Total energy over 16 μs is a few pJ.

```
SPICE Error Log: C:\Users\osama\Downloads\PROJCT\PROJCT\final.log
Gmin = 0.115292
Gmin = 0.0123794
Gmin = 0.00132923
Gmin = 0.000142725
Gmin = 1.5325e-005
Gmin = 1.6455e-006
Gmin = 1.76685e-007
Gmin = 1.89714e-008
Gmin = 2.03704e-009
Gmin = 2.18725e-010
Gmin = 2.34854e-011
Gmin = 2.52173e-012
Gmin = 2.70769e-013
Gmin = 0
Gmin stepping succeeded in finding the operating point.

iavg: AVG(i(vdd))=-3.20636e-007 FROM 0 TO 1.6e-005
pavg: AVG(v(vdd)*i(vdd))=-5.77145e-007 FROM 0 TO 1.6e-005
ppeak: MAX(v(vdd)*i(vdd))=0.000280002 FROM 0 TO 1.6e-005
etotal: INTEG(v(vdd)*i(vdd))=-9.23431e-012 FROM 0 TO 1.6e-005


Date: Thu Jul 02 18:05:58 2026
Total elapsed time: 2.452 seconds.

tnom = 27
temp = 27
```

Fig. 17: Power/energy summary.

TABLE III: Results Summary

Metric	Value
Array size	4×4 (16 bits)
Pull-down NMOS width	$W = 10$ (fixed from $W = 5$)
Supply voltage	1.8 V
Average power	$\approx$0.58 μW
Peak power	$\approx$280 μW
Total energy (16 μs)	$\approx$9.23 pJ
Layout area	$\approx$0.154 mm^2
DRC	Clean
Bits read out	Q_0–Q_3, all 16 (row-by-row)
Unstable-bit method	Repeated power-up
Quantitative BER	Not measured yet

TABLE IV: Baseline (Direct Latch) vs. Proposed (Schmitt)

Aspect	Baseline	Proposed
Readout	Direct latch	Schmitt trigger
Noise margin	Normal	Wider (hysteresis)
Pull-down sizing	$W = 5$	$W = 10$
Unstable-bit check	None	Repeated-read
Layout area	Not measured	$\approx$0.154 mm^2
Expected BER	Higher	Lower

V. Baseline vs. Proposed

A separate baseline layout was not built, so its area is left blank. BER numbers are expected trends, not yet measured; that is next.

VI. Conclusion

A small SRAM PUF with Schmitt-trigger readout was built and simulated in Electric VLSI at $V_{DD} = 1.8$ V. Fixing the pull-down width from $W = 5$ to $W = 10$ solved a real discharge problem, verified at both the single-cell and full-array level. The design showed both PUF properties: stable on repeat (Fig. 11a), different on independent runs (Fig. 11b), though the randomness here comes from solver-level numerical noise, not

an explicit mismatch model. Average power ($\approx$0.58 μW), peak power ($\approx$280 μW), and layout area ($\approx$0.154 mm^2, DRC clean) were measured and compared directly to ASCH-PUF [1].

Next steps: run post-layout (extracted) simulation; replace the solver-noise randomness with a real Monte Carlo mismatch sweep (`.step` with a random parameter on one transistor) to get an actual BER number; build a separate baseline layout for a fair area comparison.

REFERENCES

[1] Y. He, D. Li, Z. Yu, and K. Yang, "ASCH-PUF: A 'Zero' Bit Error Rate CMOS Physically Unclonable Function With Dual-Mode Low-Cost Stabilization," *IEEE Journal of Solid-State Circuits*, 2023.